Racing Line Optimization

Je Hon Tan John Papayanopoulos

December 15, 2016

Nomenclature

δ	Control point positioning parameter.
κ	Curvature of trajectory.
$a_{b,max}$	Maximum tangential acceleration due to braking.
$a_{c,max}$	Maximum centripetal acceleration before slipping
$a_{t,max}$	Maximum tangential acceleration due to throttle.
L	Length of trajectory.
s	Position along trajectory.
T	Lap time.
v	Tangential velocity of vehicle.
v_κ	Maximum tangential velocity allowed due to curvature and side-slip condition.
v_{max}	Top speed of vehicle.

1 Introduction

This project explores the problem of racing line optimization in motorsport. The racing line is the route taken by a racing car around a racetrack in order to obtain the fastest lap time. Racing lines are of key importance in racing motorsport, given the convergence of vehicle designs due to competition regulations. The choice of racing line often depends on driver experience and skill, however it is desirable to automate the finding of the optimum racing line for new racetracks, new vehicles or inexperienced drivers.

An approach using minimum trajectory curvature was first explored. The minimum curvature route is useful as it minimizes the turning and tangential acceleration inputs necessary to complete the route and hence maximizes energy efficiency (assuming no drag). This method was then modified to produce minimum time.

2 Problem Definition

The objective is to find a trajectory that minimizes the lap time around a given racetrack give a standing start, i.e. an initial velocity of 0.

2.1 Track Definition

The track was determined by taking an image of a track and implementing machine vision algorithms for finding the borders of the track. This was done by taking the image and assuming the road is the darkest part of the image (approaching black). The image is converted to a binary image that is a logical array. The image is smoothed and using morphological operations to close the image (dilate, then erode with a mask), holes are removed. Finally, the MATLAB function `bwboundaries` is called to determine all boundaries in the image, ordered in significance. The two most significant borders are selected as they are (almost certainly) the inside and outside borders of the track. A limitation of this algorithm is that it will not work with tracks that have overpasses (such as a figure-eight).

From a given track, such as that in Figure 1, the borders found were similar to that in Figure 2

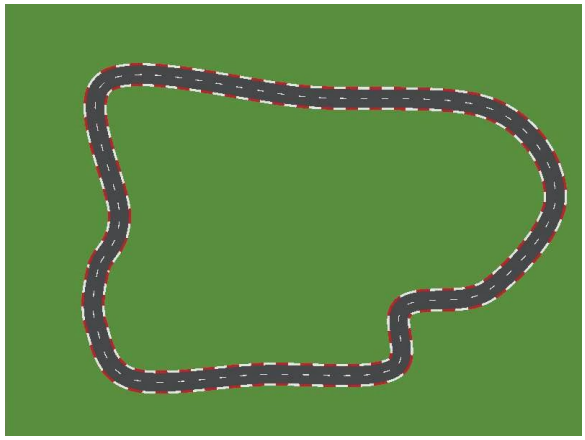

Figure 1: Example track

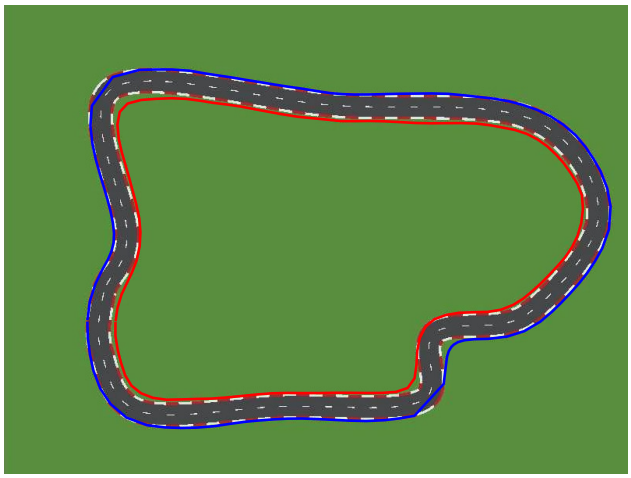

Figure 2: Borders determined on example track

2.2 Trajectory Model

To allow the optimization process to be generic given wildly different track geometries, a path coordinate system was used, where the position s along the track was used to parameterize the various functions used.

The trajectory was modeled as an interpolation between a number of

control points. Two methods of interpolation were used, a simple linear interpolation and a more 4th-order B-spline. The high order of the B-spline used was so as to obtain a smooth curvature function $\kappa(s)$. Since κ is a function of the 2nd derivative of the trajectory, a minimum of a 3rd-order B-spline is necessary for a continuous $\kappa(s)$, and a 4th order B-spline is required for a smooth $\kappa(s)$.

However it was found subsequently that computing the spline slowed the optimization process down significantly, and that good results could be obtained at a much lower computational cost by using a linear interpolation with a higher density of points. The B-spline model was thus not used in the final optimization process.

Two approaches to modifying the trajectory, or equivalently positioning the control points, were used. In the first method, named the δ-trajectory method, control points were spaced evenly along the track, and moved laterally between the inner and outer boundaries by modifying a parameter δ between 0 and 1 where 0 corresponds to a point on the inner boundary and 1 a point on the outer boundary. Figure 3 illustrates this method. This method is simple to implement and ensures to a large extent that the trajectory remains on the track, however it is poor at handling sharp turns or a number of turns spaced close to each other.

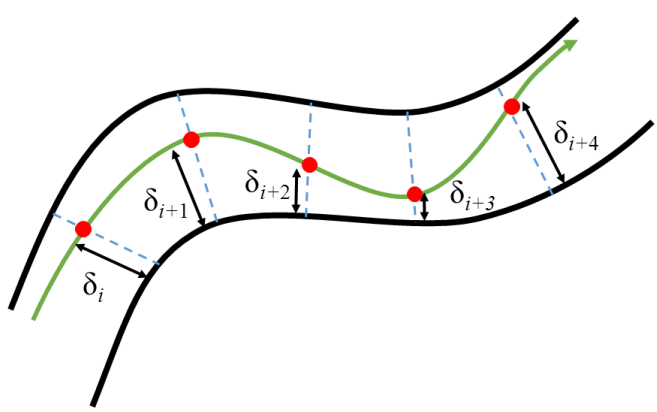

Figure 3: Control point positioning using δ parameters

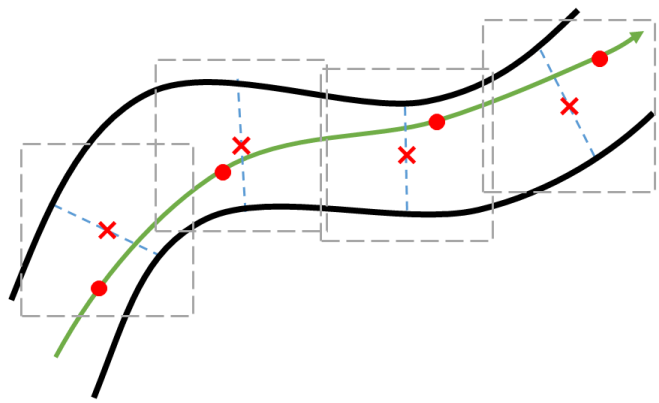

Figure 4: Free control point positioning within boundary window

In the second method, named the free-positioning method, reference points are evenly spaced around the track along the centerline, and each control point is freely positioned within a boundary window about their respective reference point. This method allows for finer control of the trajectory with fewer control points compared to the first method.

2.3 Assumptions

The racing line is influenced by a number of factors, such as vehicle dynamics, tire dynamics, etc. These factors significantly complicate attempts to optimize the racing line, hence to keep the problem tractable the following simplifying assumptions were made

1. Particle Motion. The vehicle traveling around the track is considered to be a particle, i.e. with no rotational inertia, no turning radius constrains due to wheelbase, and no vehicle dynamics.

2. Instant Acceleration. Tangential acceleration can be discontinuous allowing for sharp corners in velocity profile, with a maximum positive value a_{t_max} and maximum negative value of a_{b_max}. This is a reasonable

assumption considering that high performance racing vehicles are capable of quick acceleration and braking and that a more aggressive driving behaviour is common in motorsport. This results in a continuous but non-smooth velocity profile.

3. Constant Side-slip Condition. The vehicle is assumed to slip in a turn if the centripetal acceleration required for it to keep the turning trajectory is larger than a constant threshold a_c.

2.4 Constraints

The velocity of the vehicle is limited by several factors. Firstly it may not exceed its top speed v_{max}, which is result of air resistance, rolling resistance amongst other factors, but which is assumed to be a constant.

Secondly it may not corner at a velocity higher than is allowed by the side-slip condition, i.e a velocity where the centripetal force required to keep the vehicle on its trajectory is higher than can be provided by its tires. This is modeled as a threshold on the centripetal acceleration, and the allowable velocity calculated using the relation $v_\kappa = \sqrt{a_{c,max}/\kappa}$.

Thirdly, acceleration of the vehicle may not exceed the maximum acceleration possible using the throttle $a_{t,max}$ and the maximum negative acceleration possible due to braking $a_{b,max}$.

2.5 Optimization Problem

The various points above may be summarized as the following optimization problem

Minimize

$$T = \int_0^L \frac{1}{v(s)} ds$$

such that

$$
\begin{aligned}
v(s) &\leq v_{max} \\
v(s) &\leq v_\kappa(s) = \sqrt{\frac{a_{c,max}}{\kappa}} \\
\frac{dv}{ds} &\leq \frac{1}{v(s)} a_{t,max} \\
\frac{dv}{ds} &\leq \frac{1}{v(s)} a_{b,max}
\end{aligned}
$$

An interesting aspect of this optimization problem is its "path-dependence", i.e. that in order to satisfy all constraints the trajectory and velocity at each point depends on trajectory and velocities that occur before and after the point. This makes describing the the optimization problem using closed-form objective and penalty functions for the position of trajectory control points not possible, necessitating the use of some form of Monte Carlo method with some method of dynamically determining the feasibility of a particular solution.

3 Minimum Curvature Solution

Intuitively, a minimum curvature trajectory should be close to the optimal racing line. This is because minimizing curvature is equivalent to minimizing accelerations, both centripetal and tangential, and would thus allow a higher velocity to be maintained through corners.

3.1 Curvature Estimation

An initial approach to estimating curvature was to compute the 4th-order B-spline through the control points and obtaining the exact curvature using the first and second derivatives of the spline. However it was found that computing the spline for even a moderate number of points significantly slowed the evaluation process for each potential solution, making the Monte Carlo methods used exponentially slower. An alternative method that approximates the curvature was used instead.

The minimization of curvature can then be expressed as the optimization problem

Minimize

$$T = \int_0^L \frac{1}{\theta(s)} ds$$

To approximate the curvature at each control point the angle formed by it and its neighboring control points was found using the Law of Cosines. The control points were positioned using the δ-trajectory method to force that the trajectory remained on the track.

3.2 Implementation

δ values for each control point from the aforementioned δ-trajectory method where given to a genetic algorithm. A genetic algorithm uses "populations" of possible solution, selects the best fitting and then "breeds" them to form future "generations". This was selected for this problem as it makes sense that merging better paths should converge to the best path. This means that a genetic algorithm is naturally self reinforcing. The array of delta values produced a path for the fitness function to interpret. The fitness function was to minimize the sum of the inverse of the angle between each point and its neighbors. The genetic algorithm was capped such that the value of delta was limited to $0 \leq \delta \leq 1$.

3.3 Results

Using 43 control points around the track, and a population of 200, the fitness versus generation followed the trend shown in Figure 5. This method took nearly 700 generations before finding the optimum. Considering the complexity of the problem, this is relatively quick. The results of the minimum curvature solution are shown in Figure 6.

The velocity profile was created where the vehicle was given a maximum allowable velocity and maximum allowable tangential and normal accelerations. A method for determining the velocity profile was created and is discussed later. This velocity profiling algorithm was applied to the path of minimum curvature. The time to complete the track was found to be $T \approx 379$. This velocity profile is shown in Figure 7,where the orange line represents vehicle maximum velocity due to curvature and the blue line represents the overall maximum velocity at a given point when considering tan-

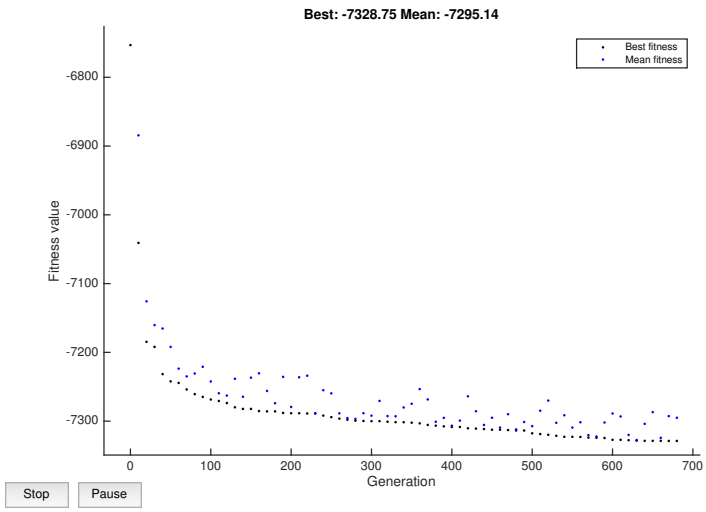

Figure 5: Fitness over generations for optimizing curvature

gential accelerations. It is clear from this profile that the velocity of the vehicle remains relatively constant.

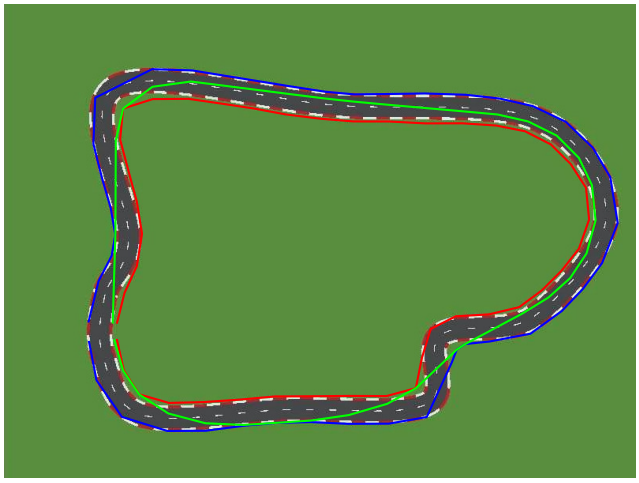

Figure 6: Minimum Curvature of Track

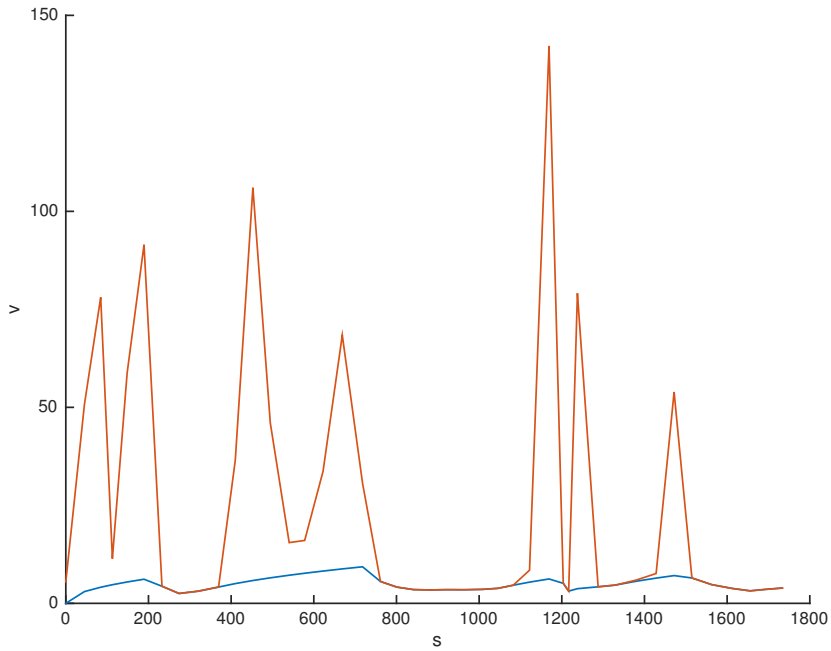

Figure 7: Velocity profile for minimum curvature path

4 Minimum Time Solution

The minimum curvature solution, while a step in the right direction, does not guarantee minimum time, as it minimizes changes in velocity. Allowing for aggressive acceleration and braking can result in a higher average velocity than the minimum curvature solution, and thus a faster lap time.

4.1 Velocity Profile Generation

A method was developed for finding the maximum velocity allowable at each point given a trajectory. This concept works well with a genetic algorithm as the control variables of the genetic algorithm must simply generate paths for this maximum velocity method to solve.

This method can be illustrated by considering an example trajectory. The vertical axis of Figure 8 is velocity and the horizontal axis is position. The plot in orange is of the maximum velocity v_κ a particle moving along that trajectory can take limited by the side-slip condition.

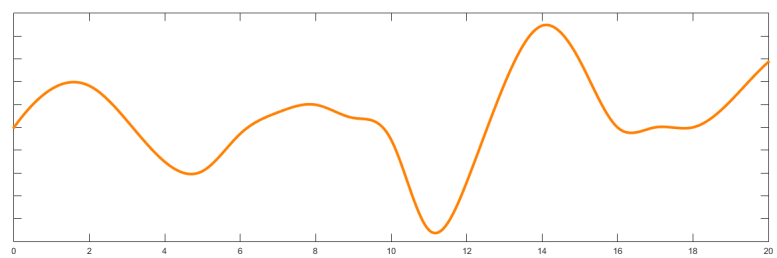

Figure 8: Maximum Velocity due to Curvature

Starting from the end of the trajectory the maximum braking acceleration condition is enforced, and the maximum velocity at the position updated as the minimum of v_κ and that yielded by maximum braking. This is shown in Figure 9 where the horizontal axis is reversed, i.e. moving left to right moves from the end of the trajectory to the start.

Finally, the process is repeated on this combined velocity profile from the start to the end of the trajectory for the positive acceleration constraint, as shown in Figure 10 (shown in green). This yields a maximum velocity profile for a given trajectory.

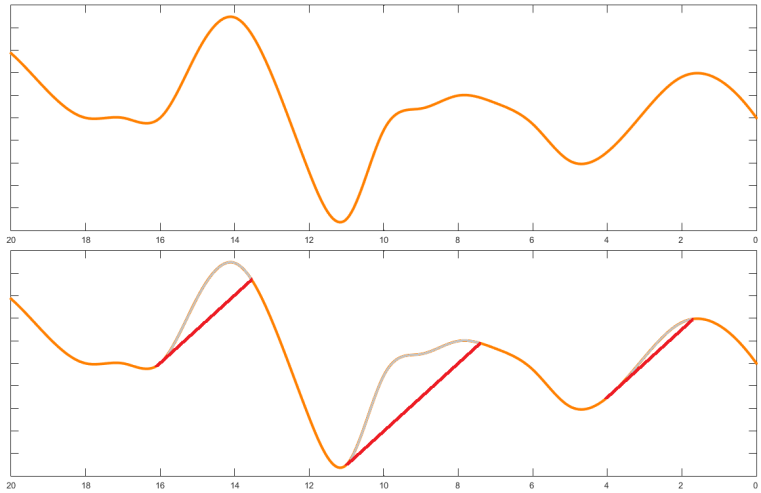

Figure 9: Maximum Velocity when considering necessary braking

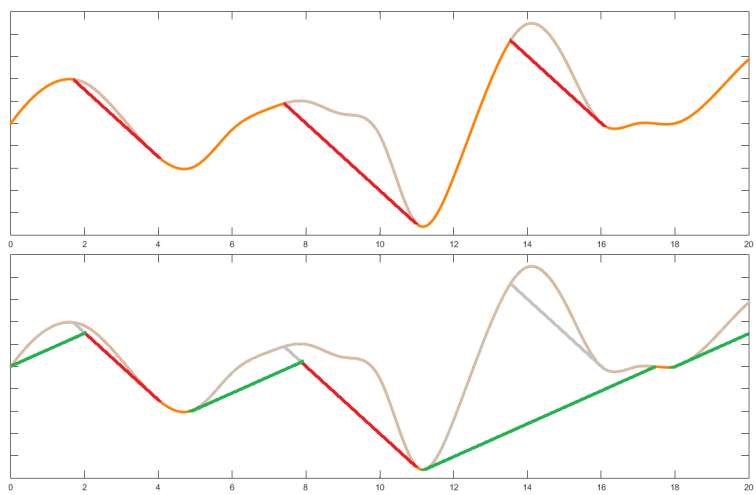

Figure 10: Maximum Velocity when considering maximum possible positive and braking acceleration

4.2 Modified Curvature Estimation

The calculation of curvature was modified from using angle to finding the actual radius of curvature at each point. This was done by calculating the radius of the circle formed by the point and two neighboring points. This was done as the radius is needed for calculating the normal acceleration at

a given point. The radius at each point was found using the below set of equations.

$$A = x_1(y_2 - y_3) - y_1(x_2 - x_3) + x_2y_3 - x_3y_2$$
$$B = (x_1^2 + y_1^2)(y_3 - y_2) + (x_2^2 + y_2^2)(y_1 - y_3) + (x_3^2 + y_3^2)(y_2 - y_1)$$
$$C = (x_1^2 + y_1^2)(x_2 - x_3) + (x_2^2 + y_2^2)(x_3 - x_1) + (x_3^2 + y_3^2)(x_1 - x_2)$$
$$D = (x_1^2 + y_1^2)(x_3y_2 - x_2y_3) + (x_2^2 + y_2^2)(x_1y_3 - x_3y_1) + (x_3^2 + y_3^2)(x_2y_1 - x_1y_2)$$
$$r = \sqrt{(B^2 + C^2 - 4AD)/(4A^2)}$$

4.3 Implementation

A genetic algorithm was used that manipulated the positions of the trajectory control points. The fitness function accepts a trajectory and implements the method for generating the velocity profile described above, and returns the lap time for the trajectory. A lower lap time is preferred by the genetic algorithm. An initial population of random trajectories on the track was used to start the optimization at a point close to the optimum solution.

The control points were positioned using the free-positioning method to allow for a greater degree of freedom to handle corners. Consequently it was necessary to ensure that the control points remained on the track. This was done by adding a large penalty to the lap time that is proportional to the number of control points off the track using found using MATLAB's `inpolygon` function.

4.4 Genetic Algorithm Variations

The genetic algorithm may be modified by using different fitness, crossover, selection and mutation functions to form various variations. Furthermore population size per generation, subpopulations, migration functions, proportion of elite children in each generation, etc. can all be modified to tweak the performance and characteristics of the algorithm. For this project, the fitness function was the velocity generation function that returned the lap time with added penalty, and the other options kept at default, save for those described below.

Firstly, the mutation function, i.e. the function that determines the difference in children from their parents, was changed from the default `Gaussian`

to `Uniform`. In the `Gaussian` function the difference in children from their parents is distributed normally, with a standard deviation σ that shrinks every generation leading to convergence. However in testing the convergence happens too quickly for our use case, leading to poor results. Also, since the position of the control points is already bounded, it is desirable to retain a good degree of randomness over generations to allow the algorithm to discover new rare solutions. Using `Uniform` instead distributes the children randomly within the specified upper and lower bounds, and is the behaviour that produces the best results.

Secondly, the population size was kept fairly small, due to the bounded nature of the control point positions, and the small size of the track picture. A large population size provides little advantage as many solutions in each generation are very similar, and increases computation time. However at the same time, due to the number of control points, the population needs to be sufficiently large to provide a meaningful coverage of potential solutions. These values were manually tuned to obtain good results.

4.5 Results

The result of running the genetic algorithm on the track when minimizing time is shown in fig. 11. In this figure, green portions of the track represent approaching max speed the vehicle travels on the track, and red the minimum speeds. It is quite clear that on straightaways the vehicle speeds up and as it approaches turns slows down to go close to minimum speed around the turn. As described above, this perfectly matches the real-life strategies. The time taken to complete this lap was $T \approx 363$, which is a significant improvement over the lap time for the minimum curvature path of $T \approx 379$. The corresponding velocity profile can be seen in Figure 12.

5 Validation and Discussion

5.1 Curvature

It is intuitive to expect the minimum curvature to favor straight lines over turns (by definition) so this result found when solving for minimum curvature is cleanly (no sharp turns) closing the loop with a great deal of linearity where possible. This can be seen most prominently in the "s-curve" portions of the

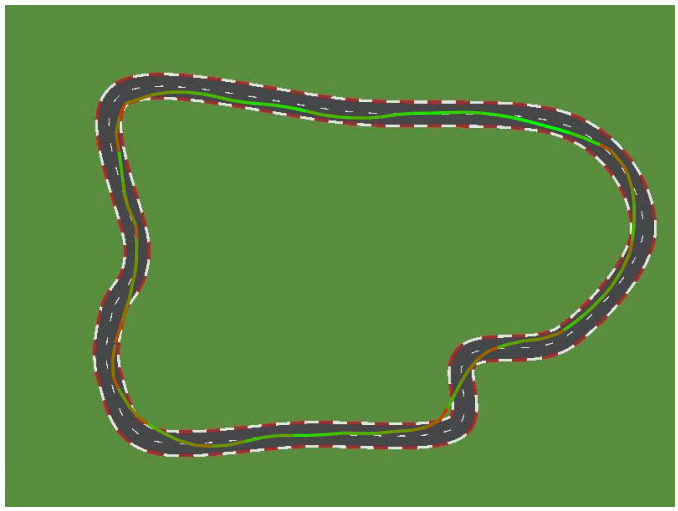

Figure 11: Optimal path for minimum track time

track where a turn exits into an opposite direction turn. "S-curves" are cut taking the inside of both such that the resulting path is very linear.

5.2 Time

While calculating precise results is a challenge, race-car drivers utilize techniques from their understanding of physics. Some expectations can therefore be compared to the results generated from this algorithm.

A straight section along the track has no curvature. Therefore, it is expected for the driver to maximize acceleration. As the driver approaches the turn, it is known that centripetal acceleration will dictate speed. Since instantaneous change in speed is not possible in the physical realm, foresight must be used and a braking must happen such that the centripetal acceleration is not exceeded when reaching the apex of the turn. It is known from kinematics that centripetal acceleration $a_c = v^2/r$, where r is the radius of curvature. Therefore, as $a_c = a_{c,max}$ is taken to be a constant, that the larger

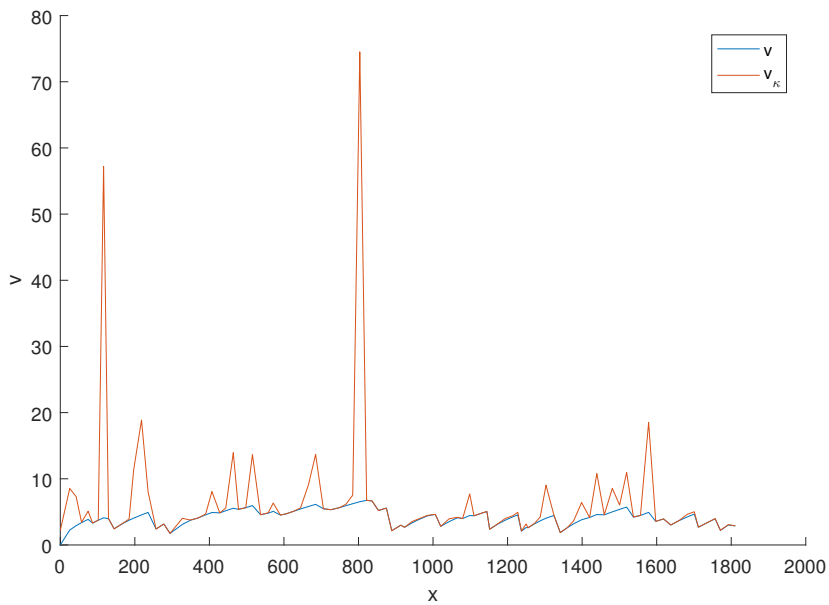

Figure 12: Optimal velocity profile for minimum track time

the r, the faster the turn can be taken. This means that the driver will be close to the outside of the track when entering and exiting a turn and close to the inside of the track in between. This minimizes curvature, allowing for maximum velocity around the turn. This should look something similar to what is shown in fig. 13. The reason it is better to not always follow the path of lowest curvature is that with the use of the gas and braking, higher speeds can be achieved by leading a turn earlier which entails moving to the outside of the track in anticipation of the turn (which increases total curvature around the track).

This method of using Genetic Algorithms has shown to be a very good way to solve the racing line problem. Some issues were run into with local minima being found as the genetic algorithm was stuck in generations that it preferred, but this was easily rectified by increasing the size of the populations which essentially increases randomness. The number of iterations required to solve this problem was found to be relatively few considering the number of control variables and each iteration only ever called the objective function once. This shows that this method for solving for the optimal racing line is

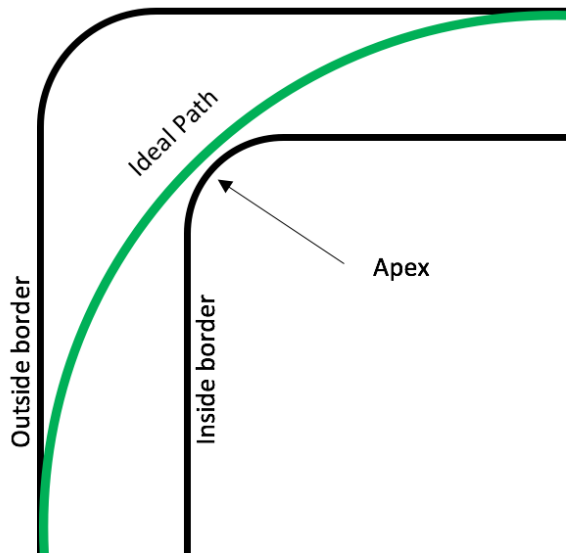

Figure 13: Path that maximizes speed around a 90° turn

not only viable, but practical.

6 Replication

This algorithm for speed was rerun on other tracks, showing that this algorithm can work for even F1 racetracks. The results were good, showing results that follow techniques as explained above and are shown in Figures 14 and 15. The vehicle slows down as it approaches and follows turns and speeds up in straightaways. It should be noted that the corner cutting is due to the lower resolution as the image is of similar size in pixels but the track is much longer. Nevertheless, this shows that this algorithm can be rerun across multiple tracks. In a setting where the algorithm would be used in a real life scenario, there should be no problem with running the algorithm for longer using very high resolution and increasing the population size for a very precise solution.

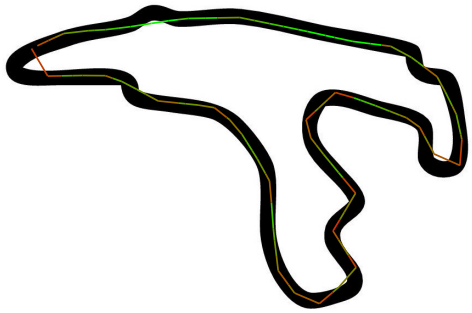

Figure 14: Optimization of the F1 Spa Track

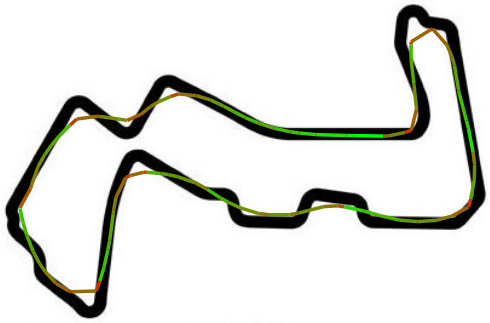

Figure 15: Optimization of the F1 Singapore Grand Prix Track

Image Sources

1. Basic Track: http://2.bp.blogspot.com/-n_JHPmowrJE/UrZcro0trpI/AAAAAAAAFV0/16-SsjIJy-Q/s1600/000036-01-25-34.jpg
2. Circuitde Spa: https://cdn.shopify.com/s/files/1/0167/3466/products/Spa-Francorchamps_Belgium_1024x1024.jpg?v=1341786630
3. Singapore Grand Prix: https://cdn.shopify.com/s/files/1/0699/0217/products/Marina-Bay-Street-Circuit-Singapore-Grand-Prix-Track-Sculpture-Full-Size-Dimensions_1024x1024.jpg?v=1471361402

Minimum Curvature Algorithm

```
clear
%read image and convert to binary
im = imread('Track.jpg');
bwim = ~im2bw(im,.4);
%remove imperfections
bwim = imclose(bwim,ones(20));
%determine boundaries
bounds = bwboundaries(bwim);
%plot track
imshow(im)
hold on
%extract the inside and outside borders
bLength = max([length(bounds{1}),length(bounds{2})]);
K = zeros(bLength-2,2);
X = zeros(bLength,2);
Y = zeros(bLength,2);
for k = 1:length(bounds)
    b = bounds{k};
    [X(1:length(b),k),Y(1:length(b),k)] = ...
    smoothPoints(b(:,1),b(:,2),20);
    K(1:length(b)-2,k) = ...
    findCurvature(X(1:length(b),k),Y(1:length(b),k));
end
Xout = X(1:length(bounds{1}),1);
Xin = X(1:length(bounds{2}),2);
Yout = Y(1:length(bounds{1}),1);
Yin = Y(1:length(bounds{2}),2);
Kout = K(1:length(bounds{1})-2,1);
Kin = K(1:length(bounds{2})-2,2);
clear('K','k','bLength','b','X','Y')

%% Initialize
%Make the inside and outside borders the same length
Xout2 = inf(length(Xin),1);
Yout2 = Xout2;
for i = 1:length(Xin)
```

```
    minDist = inf;
    for j = 1:length(Xout)
        if minDist>sqrt((Xout(j)-Xin(i)).^2+ ...
                (Yout(j)-Yin(i)).^2)
            Xout2(i) = Xout(j);
            Yout2(i) = Yout(j);
            minDist = sqrt((Xout(j)-Xin(i)).^2+ ...
                (Yout(j)-Yin(i)).^2);
        end
    end
end


%%
%resampling (take every 50th point of the bound
% boundary vector is excessively large)
resolution = 50;

Xin = Xin(1:resolution:end);
Xout2 = Xout2(1:resolution:end);
Yin = Yin(1:resolution:end);
Yout2 = Yout2(1:resolution:end);

%plot inside and outside borders
plot(Yin,Xin,'r','linewidth',2);
hold on
plot(Yout2,Xout2,'b','linewidth',2);

%%
%function handles for finding x and y coordinates
%given delta between borders
routeX = @(delta) delta(:,1).*(Xout2-Xin)+Xin;
routeY = @(delta) delta(:,1).*(Yout2-Yin)+Yin;
%set genetic algorithm parameters
options = gaoptimset('UseParallel',true,'PlotFcn', ...
        @gaplotbestf,'PlotInterval',10,...
    'PopulationSize',200);
%define fitness function handle
totCurve = @(delta) sum(abs(...
```

```
        findAngle(routeX(delta'),routeY(delta'))));
%run GA
deltaFinal = ga(totCurve,length(Xin),[],[],...
        [],[],zeros(length(Xin),1),...
    ones(length(Xin),1),[],options);
%%
%plot resulting path
figure(1)
[a] = routeX(deltaFinal');
[b] = routeY(deltaFinal');
plot(b,a,'g','linewidth',2);
```

Minimum Curvature Algorithm (Sub function)

```
function K = findAngle(X,Y)
%Create vectors of points (x1 and y1) and their
%       neighbors (x2 x3 and y2 y3)
x2 = circshift(X,-1);
x1 = X;
x3 = circshift(X,1);

y2 = circshift(Y,-1);
y1 = Y;
y3 = circshift(Y,1);

%Compute angle
K = atan2(abs((x2-x1).*(y3-y1)-(x3-x1).*(y2-y1)), ...
                (x2-x1).*(x3-x1)+(y2-y1).*(y3-y1));
%return inverse of angles
K = 1./K;

end
```

Velocity Generation Function

```
function [T,v] = findVelProfile(P,inBnd,outBnd)
% Generates maximum velocity profile

K = findCurvature(P(:,1),P(:,2));

%%
v_max = 20; % max. tangential velocity.
ac_max = .1; % max. centripetal acceleration.
at_max = .1; % max. tangential acceleration.
ab_max = -.2; % max. braking acceleration.

%%
dx = diff(P,1,1)';
dx = sqrt(dx(1,:).^2+dx(2,:).^2);
x = [0,cumsum(dx)];
dx = diff(x);

v_K = sqrt(ac_max./K); % max. vel. allowed by curvature

%%
xr = fliplr(x);
dxr = abs(diff(xr));
v_Kr = fliplr(v_K);
vr = v_Kr;
vnext = zeros(size(vr));
for i = 2:length(xr)
    vnext(i) = sqrt(vr(i-1)^2-2.*ab_max.*dxr(i-1));
    vr(i) = min(vnext(i),vr(i));
end

v = fliplr(vr);
v(1) = 0;
for i = 2:length(x)
    v(i) = min(sqrt(v(i-1)^2+2.*at_max.*dx(i-1)),v(i));
end
v = min(v,v_max);
```

```
% figure;
% hold on;
% plot(x,v);
% plot(x,v_K);

t = cumsum(dx'./v(2:end));
T = t(end);

T = T+10*penG(P,outBnd,inBnd);
end
```

Minimum Time Algorithm

```
%% Initialize track
clear
im = imread('Track.jpg');
bwim = ~im2bw(im,.4);
bwim = imclose(bwim,ones(10));
bounds = bwboundaries(bwim);
C = bwlabel(~bwim);
Centroid = regionprops(C,'centroid');
bLength = max([length(bounds{1}),length(bounds{2})]);
X = zeros(bLength,2);
Y = zeros(bLength,2);
for k = 1:length(bounds)
    b = bounds{k};
    [X(1:length(b),k),Y(1:length(b),k)] = ...
        smoothPoints(b(:,1),b(:,2),20);
end
Xout = X(1:length(bounds{1}),1);
Xin = X(1:length(bounds{2}),2);
Yout = Y(1:length(bounds{1}),1);
Yin = Y(1:length(bounds{2}),2);
clear('K','k','bLength','b','X','Y')

Xout2 = inf(length(Xin),1);
Yout2 = Xout2;
for i = 1:length(Xin)
    minDist = inf;
    for j = 1:length(Xout)
        if minDist>sqrt((Xout(j)-Xin(i)).^2+...
                (Yout(j)-Yin(i)).^2)
            Xout2(i) = Xout(j);
            Yout2(i) = Yout(j);
            minDist = sqrt((Xout(j)-Xin(i)).^2+...
                (Yout(j)-Yin(i)).^2);
        end
    end
end
```

```
%% Resample Boundaries
resolution = 100;

Xin = Xin(round(linspace(1,length(Xin),resolution)));
Xout2 = Xout2(round(...
        linspace(1,length(Xout2),resolution)));
Yin = Yin(round(linspace(1,length(Yin),resolution)));
Yout2 = Yout2(round(linspace(...
        1,length(Yout2),resolution)));

%% Genetic Algorithm

popsize = 500;

inBnd = [Xin Yin];
outBnd = [Xout2 Yout2];

routeX = @(delta) delta.*(Xout2-Xin)+Xin;
routeY = @(delta) delta.*(Yout2-Yin)+Yin;

fitfunc = @(x) findVelProfile(...
        [x(1:length(x)/2)',x(length(x)/2+1:end)'],...
    inBnd,outBnd);

% Generate random initial population
initpop = zeros(popsize,2*length(Xin));
del = rand(popsize,length(Xin));
for i = 1:popsize
    initpop(i,:) = [routeX(del(i,:)')',...
        routeY(del(i,:)')'];
end

% Generate bounds
ub = sum(initpop)./popsize+5*ones(1,2*length(Xin));
lb = sum(initpop)./popsize-5.*ones(1,2*length(Xin));

options = gaoptimset('PlotFcn',@myplotfun,...
```

```
'PlotInterval',1,'UseParallel',true,...
'PopulationSize',popsize,'InitialPopulation',...
initpop,'PopInitRange',[lb;ub],'MutationFcn',...
@mutationuniform,'FitnessScalingFcn',@fitscalingtop);

%Run GA
[P_opt,T_opt] =ga(fitfunc,2*length(Xin)...
        ,[],[],[],[],[],[],[],[],options);

[~,v] = findVelProfile([P_opt(1:length(P_opt)/2)',...
        P_opt(length(P_opt)/2+1:end)'],inBnd,outBnd);
figure
hold on;
imshow(im);
plotTrack(P_opt(1:length(P_opt)/2),...
        P_opt(length(P_opt)/2+1:end),v);
```

Modified Find Curvature Algorithm

```
function K = findCurvature(X,Y)
        %Find each points neighbors
    x1 = circshift(X,-1);
    x2 = X;
    x3 = circshift(X,1);

    y1 = circshift(Y,-1);
    y2 = Y;
    y3 = circshift(Y,1);
        %Set up equations
    A = x1.*(y2-y3)-y1.*(x2-x3)+x2.*y3-x3.*y2;
    B = (x1.^2+y1.^2).*(y3-y2)+(x2.^2+y2.^2).*...
        (y1-y3)+(x3.^2+y3.^2).*(y2-y1);
    C = (x1.^2+y1.^2).*(x2-x3)+(x2.^2+y2.^2).*...
        (x3-x1)+(x3.^2+y3.^2).*(x1-x2);
    D = (x1.^2+y1.^2).*(x3.*y2-x2.*y3)+...
        (x2.^2+y2.^2).*(x1.*y3-x3.*y1)+...
                (x3.^2+y3.^2).*(x2.*y1-x1.*y2);
    %Solve for curvature
    r = sqrt((B.^2+C.^2-4.*A.*D)./(4.*A.^2));
    K=1./r;
end
```